

$\mathbb{F}_2\mathbb{Z}$

1

1

July 27, 2026

Abstract

Albert: Should the name be $\mathbb{F}_2\mathbb{F}$?

Contents

1	Introduction	2
1.1	Acknowledgements	2
2	Preliminaries	2
3	$\mathbb{F}_2\mathbb{Z}$	3
3.1	Constrained codes over dual fields Albert: not sure dual is a good word: the two fields are just “different”, while dual things tend to have a close relation between each other	3
3.2	Tensor decomposition, interleaved codes, and products of constrained codes	4
3.3	The main $\mathbb{F}_2\mathbb{Z}$ IOR	5
4	Virtualization of $\mathbb{F}_2$-linear combinations lifted to integers	6
5	R1CS constraints over polynomial rings with ideal membership	11
6	Case-study: SHA-256 hashing	11
6.1	Arithmetization over $\mathbb{Z}[X]$	11
6.1.1	SHA-256 compression overview	11
6.1.2	Our arithmetization	12
6.2	End-to-end IOP	12
6.3	Experiments	12

7	Case-study: Keccak-f[1600] hashing	13
7.1	Arithmetization over $\mathbb{Z}[X]$	13
7.1.1	Keccak-f[1600] overview	13
7.1.2	Our arithmetization	15
7.2	End-to-end IOP	17
7.3	Experiments	17
A	Ring switching via Galois orbits	17
B	[Old appendix by Ulrich] Galois FRI-Binius	18
C	Bilinear Embeddings	20
C.1	Background	20
C.2	Structure	20
C.3	Properties	23
C.4	Constructions	23
C.4.1	Trace embedding	23
C.4.2	Coefficient projection	25

1 Introduction

1.1 Acknowledgements

2 Preliminaries

Given a prime q we let $\pi_q : \mathbb{Z} \rightarrow \mathbb{F}_q$ be the ring homomorphism mapping an integer to its reduction modulo q . We further let π_0 denote the identity map on $\mathbb{Z}$, so that $\pi_0(x) = x$ for all $x \in \mathbb{Z}$. We let $\mathbb{F}_0 = \mathbb{Z}$.

Given a finite field $\mathbb{F}$ of characteristic q , let $a(X) \in \mathbb{Z}[X]$ be an irreducible polynomial over $\mathbb{F}_q$ [Albert: yes?](#), so that $\mathbb{F} = \mathbb{Z}[X]/(q, a(X))$ [Albert: consider isomorphism, or assume all finite fields are represented like this](#). Let $\psi_{\mathbb{F}} : \mathbb{Z}[X] \rightarrow \mathbb{F}$ be the canonical homomorphism from $\mathbb{Z}[X]$ to $\mathbb{F}$ obtained by reducing modulo the ideal $(q, a(X))$, i.e. by reducing coefficients modulo q and then reducing the resulting polynomial modulo $a(X)$.

Interactive Oracle Proofs of Proximity (IOPP) Interleaved codes

Interactive Oracle Reductions (IOR)

3 $\mathbb{F}2\mathbb{Z}$

3.1 Constrained codes over dual fields **Albert: not sure dual is a good word: the two fields are just “different”, while dual things tend to have a close relation between each other**

Let $\ell_0 \geq 0$ be a dimension parameter, and $m \geq 0$ a blocklength parameter. Let $\mathbb{F}$ be a field (not necessarily a prime field), and let $\mathcal{C} \subseteq \mathbb{F}^m$ be a linear code of dimension ℓ_0 and blocklength m over $\mathbb{F}$. Let $k \geq 0$ be a public input length parameter, and let $Q \in \mathbb{F}[Y_1, \dots, Y_k, Z_1, \dots, Z_\ell]$ be a constraint polynomial, of arbitrary degree, with coefficients in $\mathbb{F}$ and on $k + \ell$ variables. Further, let $\ell \geq \ell_0$ be a witness length parameter, and let $P : \mathbb{F}^\ell \rightarrow \mathbb{F}^{\ell_0}$ be an arbitrary map (looking ahead, for us P will be a packing map that packs elements from $\mathbb{F}_2$ into elements of a larger binary field. Consider the following relation:

$$\mathcal{R}_{\mathcal{C}, \mathbb{F}, P}^Q := \left\{ \left(\begin{array}{l} \mathbf{x} = \mathbf{y} \in \mathbb{F}^k, \\ \mathbf{y} = \mathbf{f} \in \mathbb{F}^m, \\ \mathbf{w} = \mathbf{f} \in \mathbb{F}^\ell \end{array} \right) \mid \begin{array}{l} \text{Enc}_{\mathcal{C}}(P(\mathbf{f})) = \mathbf{f} \\ \wedge Q(\mathbf{y}, \mathbf{f}) = 0 \end{array} \right\}$$

When Q and $\mathbf{y}$ are such that $\mathbf{y} = (\mathbf{v}, \mu) \in \mathbb{F}^\ell \times \mathbb{F}$ and

$$Q(\mathbf{y}, \mathbf{f}) = \langle \mathbf{v}, \mathbf{f} \rangle = \sum_{i \in [\ell]} \mathbf{v}_i \cdot \mathbf{f}_i,$$

we denote $\mathcal{R}_{\mathcal{C}, \mathbb{F}, P}^Q$ by $\mathcal{R}_{\mathcal{C}, \mathbb{F}, P}^{\text{lin}}$.

Next let $\mathbb{F}_{2^n}$ be a finite field of characteristic 2, and let $\mathbb{F}_q$ be a finite field of prime order q or $\mathbb{Q}$. For intuition, $\mathbb{F}_{2^n}$ is the field over which we will commit our witness, and $\mathbb{F}_q$ is the field over which we prove linear relations on the witness. We let

$$\pi_2 : \mathbb{Z} \rightarrow \mathbb{F}_2 \subseteq \mathbb{F}_{2^n}, \quad \pi_q : \mathbb{Z} \rightarrow \mathbb{F}_q,$$

be reduction mod 2 and q maps. We further let

$$\pi_2^{-1} : \mathbb{F}_2 \rightarrow \{0, 1\} \subseteq \mathbb{Z}, \quad \pi_q^{-1} : \mathbb{F}_q \rightarrow [0 .. q - 1] \subseteq \mathbb{Z},$$

be maps such that $\pi_2(\pi_2^{-1}(x)) = x$ and $\pi_q(\pi_q^{-1}(x)) = x$ for all x . We remark that one could choose any other such “inverse” maps, e.g. for π_q^{-1} one may prefer to use centered representatives.

Assume $|\mathbb{F}_{2^n}| = 2^n$. Let

$$\beta_1, \dots, \beta_n \in \mathbb{F}_{2^n}$$

be an $\mathbb{F}_2$ -base of $\mathbb{F}_{2^n}$ seen as an $\mathbb{F}_2$ -vector space of dimension n .

Given a vector $\mathbf{u} \in \mathbb{F}_2^m$, with 2^n dividing m , we define

$$\text{Pack}_n(\mathbf{u}) \in \mathbb{F}_{2^n}^{m/n}$$

as the vector whose j -th entry is

$$\text{Pack}_n(\mathbf{u})_j = \mathbf{u}_{(j-1) \cdot n + 1} \cdot \beta_1 + \dots + \mathbf{u}_{(j-1) \cdot n + n - 1} \cdot \beta_{n-1} + \mathbf{u}_{j \cdot n} \cdot \beta_n \in \mathbb{F}_{2^n}.$$

Let $\mathcal{C} \subseteq \mathbb{F}_{2^n}^m$ be a linear code of dimension $\ell/2^n$ and blocklength m over $\mathbb{F}_{2^n}$. Define

$$\mathcal{R}_{\mathcal{C}, \mathbb{F}_{2^n}, \mathbb{F}_q, \text{Pack}_n}^{\text{lin-dual}} := \left\{ \left(\begin{array}{l} \mathbf{x} = (\mu, \mathbf{v}) \in \mathbb{F}_q \times \mathbb{F}_q^\ell, \\ \mathbf{y} = f \in \mathbb{F}_{2^n}^m, \\ \mathbf{w} = \mathbf{f} \in \{0, 1\}^\ell \subseteq \mathbb{Z}^\ell \end{array} \right) \middle| \begin{array}{l} \text{Enc}_{\mathcal{C}}(\text{Pack}_n(\pi_2(\mathbf{f}))) = f \\ \wedge \langle \pi_q(\mathbf{f}), \mathbf{v} \rangle = \mu \end{array} \right\}$$

3.2 Tensor decomposition, interleaved codes, and products of constrained codes

We fix a tensor decomposition of vectors $\mathbf{v} \in \mathbb{F}_{2^n}^\ell$ as

$$\mathbf{v} = \mathbf{v}^{(1)} \otimes \mathbf{v}^{(2)} = \left(\mathbf{v}_i^{(1)} \cdot \mathbf{v}_j^{(2)} \right)_{i \in [\ell_1], j \in [\ell_2]}$$

for some $\mathbf{v}^{(1)} \in \mathbb{F}_q^{\ell_1}, \mathbf{v}^{(2)} \in \mathbb{F}_q^{\ell_2}$, with $\ell_1 \cdot \ell_2 = \ell$. Note that the trivial tensor decomposition is admitted, i.e. we may effectively not decompose $\mathbf{v}$ by choosing $\ell_1 = \ell, \ell_2 = 1, \mathbf{v}^{(1)} = \mathbf{v}$, and $\mathbf{v}^{(2)} = (1)$.

We assume $\mathcal{C} = \mathcal{C}_1^{\equiv \ell_2}$ is the ℓ_2 -dimensional interleaving of a code $\mathcal{C}_1 \subseteq \mathbb{F}_{2^n}^{m/\ell_2}$ of dimension $\ell_1/2^n$ and blocklength m/ℓ_2 . We then index the entries of $\mathbf{f}$ as $\mathbf{f} = (\mathbf{f}_{ij})_{i \in [\ell_1], j \in [\ell_2]}$. Note that

$$\langle \pi_q(\mathbf{f}), \mathbf{v} \rangle = \sum_{j \in [\ell_2]} \left(\sum_{i \in [\ell_1]} \pi_q(\mathbf{f}_{ij}) \cdot \mathbf{v}_i^{(1)} \right) \cdot \mathbf{v}_j^{(2)} = \sum_{j \in [\ell_2]} \langle \pi_q(\mathbf{f}_{*j}), \mathbf{v}^{(1)} \rangle \cdot \mathbf{v}_j^{(2)},$$

where $\mathbf{f}_{*j} = (\mathbf{f}_{ij})_{i \in [\ell_1]}$.

Define the following bivariate polynomial:

$$P(Y, Z) = (Y - 1) \cdot Z + 1$$

Note $P(Y, 0) = 1$ and $P(Y, 1) = Y$. Consider the relation $\mathcal{R}_{\mathbb{F}_{2^n}, \text{Pack}_n}^Q$ with underlying code $\mathcal{C}_1$ and

$$Q(Y_1, \dots, Y_{\ell_1+1}, Z_1, \dots, Z_{\ell_1}) = \prod_{i=1}^{\ell_1} P(Y_i, Z_i) - Y_{\ell_1+1}.$$

Precisely,

$$\mathcal{R}_{\mathcal{C}_1, \mathbb{F}_{2^n}, \text{Pack}_n}^Q = \left\{ \left(\begin{array}{l} \mathbf{x} = (\mathbf{y}, z) \in \mathbb{F}_{2^n}^{\ell_1} \times \mathbb{F}_{2^n}, \\ \mathbf{y} = f \in \mathbb{F}_{2^n}^{m/\ell_2}, \\ \mathbf{w} = \mathbf{f} \in \mathbb{F}_{2^n}^{\ell_1} \end{array} \right) \middle| \begin{array}{l} \text{Enc}_{\mathcal{C}_1}(\text{Pack}_n(\mathbf{f})) = f \\ \wedge \prod_{i=1}^{\ell_1} ((\mathbf{y}_i - 1) \cdot \mathbf{f}_i + 1) = z \end{array} \right\}$$

We assume prover and verifier have access to an IOR Π_{red} that reduces $\mathcal{R}_{\mathcal{C}_1, \mathbb{F}_{2^n}, \text{Pack}_n}^Q$ to $\mathcal{R}_{\mathcal{C}_1, \mathbb{F}_{2^n}, \text{Pack}_n}^{\text{lin}}$. We assume Π_{red} works so that the verifier never queries y [Albert: Would be better to say upfront what this protocol is in our mental model, and then make abstract assumptions like this, which look arbitrary otherwise. Same applies below.](#)

Further, we consider the ℓ_2 -fold parallel composition $\Pi_{\text{red}}^{\ell_2}$ of Π_{red} , and we assume that given ℓ_2 triples

$$\left((y^{(j)}, \mu^{(j)}), g^{(j)}, h^{(j)} \right)_{j \in [\ell_2]} \in \left(\mathcal{R}_{\mathcal{C}_1, \mathbb{F}_{2^n}, \text{Pack}_n}^Q \right)^{\ell_2}$$

the IOR $\Pi_{\text{red}}^{\ell_2}$ outputs

$$\left((v, \mu_j) \in \mathbb{F}_{2^n}^{\ell_1} \times \mathbb{F}_{2^n}, g^{(j)}, h^{(j)} \right)_{j \in [\ell_2]} \in \left(\mathcal{R}_{\mathcal{C}_1, \mathbb{F}_{2^n}, \text{Pack}_n}^{\text{lin}} \right)^{\ell_2},$$

for some $v \in \mathbb{F}_{2^n}^{\log \ell_1}$ and some $\mu_j \in \mathbb{F}_{2^n}$ for $j \in [\ell_2]$. In words, $\Pi_{\text{red}}^{\ell_2}$ outputs ℓ_2 linear claims for the input witnesses $h^{(j)}$ behind the input oracles $g^{(j)}$, all *on the same vector of coefficients* v . [Albert: We could be more abstract with this IOR, but I find it difficult to tie it with the protocol in the next section then. Maybe better to write a remark](#)

We describe one such $\Pi_{\text{red}}^{\ell_2}$ in ??, where v is a vector of the form $v = (\text{eq}(b, r))_{b \in \{0,1\}^{\log \ell_1}}$, for some $r \in \mathbb{F}_{2^n}^{\log \ell_1}$.

3.3 The main $\mathbb{F}_2\mathbb{Z}$ IOR

We provide an IOPP for $\mathcal{R}_{\mathcal{C}, \mathbb{F}_{2^n}, \mathbb{F}_q, \text{Pack}_n}^{\text{lin-dual}}$ next.

Construction 3.1 (IOPP for $\mathcal{R}_{\mathcal{C}, \mathbb{F}_{2^n}, \mathbb{F}_q, \text{Pack}_n}^{\text{lin-dual}}$). The prover and verifier receive $x = (\mu, v) \in \mathbb{F}_q \times \mathbb{F}_q^\ell$ and $y = f \in \mathbb{F}_{2^n}^m$. The prover receives $w = f \in \{0, 1\}^\ell \subseteq \mathbb{Z}^\ell$ such that $(x, y, w) \in \mathcal{R}_{\mathcal{C}, \mathbb{F}_{2^n}, \mathbb{F}_q, \text{Pack}_n}^{\text{lin-dual}}$. We assume the tensor split from the previous Section ??, and adopt its notation. Let $g \in \mathbb{F}_{2^n}$ be a nonzero element from $\mathbb{F}_{2^n}$ with sufficiently large multiplicative order, to be determined later.

1. The prover sends $\mu_1, \dots, \mu_{\ell_2} \in \mathbb{Z}$ such that $\langle f_{*j}, \pi_q^{-1}(v^{(1)}) \rangle = \mu_j$.
2. The verifier checks that $\sum_{j \in [\ell_1]} \pi_q(\mu_j) \cdot v_j^{(2)} = \mu$ over $\mathbb{F}_q$. [Albert: We could have the prover commit to the \$\mu_j\$ and have this check proved by the PCS \(a PCS that can prove linear constraints like WHIR\). This pays off for very large \$\ell\$.](#)
3. Prover and verifier define $\gamma_i = \pi_q^{-1}(v_i^{(1)}) \in [0..q-1] \subseteq \mathbb{Z}$ for $i \in [\ell_1]$, and set to prove the following claim:

$$\prod_{i \in [\ell_1]} g^{\gamma_i \cdot f_{ij}} = g^{\mu_j} \in \mathbb{F}_{2^n}, \quad \text{for all } j \in [\ell_2].$$

Note that this claim is equivalent to

$$\prod_{i \in [\ell_1]} ((g^{\gamma_i} - 1) \cdot \mathbf{f}_{ij} + 1) = g^{\mu_j} \quad \in \mathbb{F}_{2^n}, \quad \text{for all } j \in [\ell_2].$$

To prove the above claim, prover and verifier engage in the IOR $\Pi_{\text{red}}^{\ell_2}$ with input **Albert**: The reason I'm leaving this IOR abstract here is that I think there may be better ways to prove the above claim than a grand product GKR, so I want to write it modular

$$(\mathbf{x}_j = ((g^{\gamma_i})_{i \in [\ell_1]}, g^{\mu_j}), \mathbf{y}_j = f_{*j}, \mathbf{w}_j = \pi_2(\mathbf{f}_{*j}))_{j \in [\ell_2]},$$

where in the honest case $f_{*j} = \text{Enc}_{C_1}(\text{Pack}_n(\pi_2(\mathbf{f}_{*j}))) \in \mathbb{F}_{2^n}^{m/\ell_2}$. At the end of this IOR, they are left with the following claim to prove:

$$((\mathbf{v}, \mu'_j), f_{*j}, \pi_2(\mathbf{f}_{*j}))_{j \in [\ell_2]} \in \left(\mathcal{R}_{C_1, \mathbb{F}_{2^n}, \text{Pack}_n}^{\text{lin}} \right)^{\ell_2} \quad (1)$$

for some vector $\mathbf{v}$ and field elements μ'_j output during the IOR $\Pi_{\text{red}}^{\ell_2}$.

4. Prover and verifier engage in a IOPP to certify the claims (1).

Theorem 3.2. *Albert: Completeness and RBR knowledge soundness of Construction 3.1*

Proof. It follows from [Theorem 4.3](#) by taking $\ell'_1 = \ell_1$ and taking M to be the identity matrix. $\square$

We now describe a GKR-based IOR reducing $(\mathcal{R}_{C_1, \mathbb{F}_{2^n}}^Q)^{\ell_2}$ to $(\mathcal{R}_{C_1, \mathbb{F}_{2^n}}^{\text{eval}})^{\ell_2}$ of the required form.

Construction 3.3 (Grand product GKR). **Albert:** Todo: it's a standard grand product GKR so far – though there's lots of implementation-level optimizations it can undergo

Ulrich: Maybe to just write verbally, that we will use GKR for the grand product. And defer the explicit protocol description to the appendix or any later section? At this point it is maybe more important to clarify how useful the construction is because of packed commitment scheme over $\mathbb{F}_2$. **Albert:** Sounds good

4 Virtualization of $\mathbb{F}_2$ -linear combinations lifted to integers

In this section we expand relation $\mathcal{R}_{C, \mathbb{F}_{2^n}, \mathbb{F}_q, \text{Pack}_n}^{\text{lin-dual}}$ to impose, instead of constraints of the form $\text{Enc}_C(\text{Pack}_n(\pi_2(\mathbf{f}))) = f$ and $\langle \pi_q(\mathbf{f}), \mathbf{v} \rangle = \mu$, constraints as $\text{Enc}_C(\text{Pack}_n(\pi_2(\mathbf{f}))) = f$ and

$$\langle \pi_q(\pi_2^{-1}(M \cdot \pi_2(\mathbf{f}))) \rangle = \mu$$

for a matrix $M \in \mathbb{F}_2^{\ell' \times \ell}$. Precisely, given a new parameter ℓ' , we define

$$\mathcal{R}_{\mathcal{C}, \mathbb{F}_{2^n}, \mathbb{F}_q, \text{Pack}_n}^{\text{lin-virtual}} := \left\{ \left(\begin{array}{l} \mathbf{x} = (\mu, \mathbf{v}, M) \in \mathbb{F}_q \times \mathbb{F}_q^{\ell'} \times \mathbb{F}_2^{\ell' \times \ell}, \\ \mathbf{y} = f \in \mathbb{F}_{2^n}^m, \\ \mathbf{w} = \mathbf{f} \in \{0, 1\}^\ell \subseteq \mathbb{Z}^\ell \end{array} \right) \middle| \begin{array}{l} \text{Enc}_{\mathcal{C}}(\text{Pack}_n(\pi_2(\mathbf{f}))) = f \\ \wedge \langle \pi_q(\pi_2^{-1}(M \cdot \pi_2(\mathbf{f})), \mathbf{v}) = \mu \end{array} \right\}$$

The relation has the tag “lin(ear)-virtual” to reference the intuition that $\mathcal{R}_{\mathcal{C}, \mathbb{F}_{2^n}, \mathbb{F}_q, \text{Pack}_n}^{\text{lin-virtual}}$ expresses a linear constraint on the vector $\mathbf{h} = \pi_q(\pi_2^{-1}(M \cdot \pi_2(\mathbf{f}))) \in \mathbb{F}_q^{\ell'}$, which is not committed directly: only $\pi_2(\mathbf{f})$ is. Note that $\mathbf{h}$ is a linear expression over $\mathbb{F}_{2^n}$ of the elements in $\pi_2(\mathbf{f})$, lifted to the integers, and then projected to $\mathbb{F}_q$.

Albert: Would be good to add more detail in this paragraph The relation $\mathcal{R}_{\mathcal{C}, \mathbb{F}_{2^n}, \mathbb{F}_q}^{\text{lin-virtual}}$ allows to, for example, express linear constraints over $\mathbb{F}_q$ of the XOR of witness elements, while only committing (over $\mathbb{F}_{2^n}$) to the original elements. The same applies with other bit operations such as AND and Majority, by noting first that, given integer bits $x, y, z \in \{0, 1\}$: **Albert:** Should reference some identity in preliminaries, or explain more here

$$\begin{aligned} \pi_q(x \text{ AND } y) &= 2^{-1} \cdot \pi_q(x + y - \pi_2^{-1}(\pi_2(x) + \pi_2(y))), \\ \pi_q(\text{Maj}(x, y, z)) &= 2^{-1} \cdot \pi_q(x + y + z - \pi_2^{-1}(\pi_2(x) + \pi_2(y) + \pi_2(z))), \end{aligned}$$

where 2^{-1} denotes the inverse of 2 modulo q . Hence AND and Majority can be expressed over $\mathbb{F}_q$ as linear expressions of a vectors of the form $\pi_q(\pi_2^{-1}(M \cdot \pi_2(\mathbf{f})))$, with $\mathbf{f}$ being a vector of integer bits containing only the original elements on which we apply the AND and Majority operations.

We next provide an IOPP for $\mathcal{R}_{\mathcal{C}, \mathbb{F}_{2^n}, \mathbb{F}_q, \text{Pack}_n}^{\text{lin-virtual}}$ for specific shapes of M . Similarly as in ??, we assume a tensor structure $\ell = \ell_1 \cdot \ell_2$ for $\mathbf{v}$, and write $\mathbf{f} = (\mathbf{f}_{*j})_{j \in [\ell_2]} \in \{0, 1\}^{\ell_1 \times \ell_2}$. We then let ℓ'_1 be such that $\ell' = \ell'_1 \times \ell_2$ and assume $M = \text{Id} \otimes M_0$ where M_0 is a $\ell'_1 \times \ell_1$ matrix with entries in $\mathbb{F}_2$, and Id is the $\ell_2 \times \ell_2$ matrix. In words, M is the $\ell' \times \ell = \ell'_1 \cdot \ell_2 \times \ell_1 \cdot \ell_2$ diagonal block matrix with M_0 as diagonal blocks. Then

$$M \cdot \pi_2(\mathbf{f}) = (M_0 \cdot \pi_2(\mathbf{f}_{*j}))_{j \in [\ell_2]} \in (\mathbb{F}_2^{\ell'_1})^{\ell_2}. \quad (2)$$

Construction 4.1 (IOPP for $\mathcal{R}_{\mathcal{C}, \mathbb{F}_{2^n}, \mathbb{F}_q}^{\text{lin-virtual}}$). The prover and verifier receive $\mathbf{x} = (\mu, \mathbf{v}, M) \in \mathbb{F}_q \times \mathbb{F}_q^{\ell'} \times \mathbb{F}_2^{\ell' \times \ell}$ and $\mathbf{y} = f \in \mathbb{F}_{2^n}^m$. The prover receives $\mathbf{w} = \mathbf{f} \in \{0, 1\}^\ell \subseteq \mathbb{Z}^\ell$ such that $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}_{\mathcal{C}, \mathbb{F}_{2^n}, \mathbb{F}_q}^{\text{lin-virtual}}$. We assume M is of the form described above.

Define $\mathbf{h} = \pi_2^{-1}(M \cdot \pi_2(\mathbf{f})) \in \{0, 1\}^{\ell'} \subseteq \mathbb{Z}^{\ell'}$. By (2),

$$\mathbf{h} = (\mathbf{h}_{*j})_{j \in [\ell_2]} = (\pi_2^{-1}(M_0 \cdot \pi_2(\mathbf{f}_{*j})))_{j \in [\ell_2]},$$

with $\mathbf{h}_{*j} \in \{0, 1\}^{\ell'_1}$. Let $f = \text{Enc}_{\mathcal{C}}(\text{Pack}_n(\pi_2(\mathbf{f}))) \in \{0, 1\}^m \subseteq \mathbb{Z}^m$.

1. Prover and verifier execute [Construction 3.1](#) with inputs

$$\mathbf{x}' = (\mu, \mathbf{v}), \quad \mathbf{y}' = f, \quad \mathbf{w}' = \mathbf{h} = \pi_2^{-1}(M \cdot \pi_2(\mathbf{f})) \in \{0, 1\}^{\ell'} \subseteq \mathbb{Z}^{\ell'}.$$

until Step 3 where they are meant to use the IOR Π_{red} . Note that until that point, the verifier has not made any queries to $\mathbf{y}'$. So, in terms of completeness, it does not matter, yet, that f is not the encoding of $\pi_2(\mathbf{h}) = M \cdot \pi_2(\mathbf{f})$.

2. Prover and verifier engage in $\Pi_{\text{red}}^{\ell_2}$ with inputs

$$\left(\mathbf{x}_j = ((g^{\gamma_i})_{i \in [\ell'_1]}, g^{\mu_j}), \mathbf{y}_j = f_{*j}, \mathbf{w}_j = \pi_2(\mathbf{h}_{*j}) = M_0 \cdot \pi_2(\mathbf{f}_{*j}) \right)_{j \in [\ell_2]},$$

where $f_{*j} = \text{Enc}_{C_1}(\text{Pack}_n(\pi_2(\mathbf{f}_{*j})))$. At the end of $\Pi_{\text{red}}^{\ell_2}$ they are left with claims

$$((\mathbf{v}, \eta_j), \mathbf{y}_j, \mathbf{w}_j)_{j \in [\ell_2]} \in \left(\mathcal{R}_{C_1, \mathbb{F}_{2^n}, \text{Pack}_n}^{\text{lin}} \right)^{\ell_2}, \quad (3)$$

for some $\mathbf{v} \in \mathbb{F}_{2^n}^{\ell'_1}$ and $\eta_j \in \mathbb{F}_{2^n}$. Again, since we assume the verifier never queries $\mathbf{y}_j$ during Π_{red} , completeness holds for honest provers, even if $\mathbf{y}_j$ is not the encoding of $\mathbf{w}_j$.

Note that (3) holds if and only if, for all $j \in [\ell_2]$,

$$\begin{aligned} \eta_j &= \sum_{i \in [\ell'_1]} \mathbf{v}_i \cdot \pi_2(\mathbf{h}_{ij}) = \sum_{i \in [\ell'_1]} \mathbf{v}_i \cdot \sum_{t \in [\ell_1]} M_{0it} \cdot \pi_2(\mathbf{f}_{tj}) \\ &= \sum_{t \in [\ell_1]} \left(\sum_{i \in [\ell'_1]} M_{0it} \cdot \mathbf{v}_i \right) \cdot \pi_2(\mathbf{f}_{tj}) = \langle \mathbf{u}_j, \pi_2(\mathbf{f}_{*j}) \rangle, \end{aligned}$$

where $\mathbf{u} \in \mathbb{F}_{2^n}^{\ell'_1}$ is the vector whose t -th entry is

$$\mathbf{u}_t = \langle \mathbf{v}, M_{0*t} \rangle \in \mathbb{F}_{2^n},$$

and $M_{0*t} \in \mathbb{F}_{2^n}^{\ell'_1}$ denotes the t -th column of M_0 , for $t \in [\ell_1]$,

Note that, if $\Pi_{\text{red}}^{\ell_2}$ is as in ??, then $\mathbf{v} = (\text{eq}(\mathbf{b}, \mathbf{r}))_{\mathbf{b} \in \{0, 1\}^{\log \ell'_1}}$ for some $\mathbf{r} \in \mathbb{F}_{2^n}^{\log \ell'_1}$, and then $\mathbf{u}_t = \text{mle}[M_{0*t}](\mathbf{r}) \in \mathbb{F}_{2^n}$.

3. If the prover is honest, we have that

$$((\mathbf{u}, \eta_j), \mathbf{y}_j = \text{Enc}_{C_1}(\text{Pack}_n(\pi_2(\mathbf{f}_{*j}))), \pi_2(\mathbf{f}_{*j}))_{j \in [\ell_2]} \in \left(\mathcal{R}_{C_1, \mathbb{F}_{2^n}, \text{Pack}_n}^{\text{lin}} \right)^{\ell_2}. \quad (4)$$

Prover and verifier use the Galois ring-switch [Construction A.1](#) to reduce (4) to a claim of the form.

$$((\mathbf{u}', \kappa_j), \mathbf{y}_j = \text{Enc}_{C_1}(\text{Pack}_n(\pi_2(\mathbf{f}_{*j}))), \text{Pack}_n(\pi_2(\mathbf{f}_{*j})))_{j \in [\ell_2]} \in \left(\mathcal{R}_{C_1, \mathbb{F}_{2^n}, \text{id}}^{\text{lin}} \right)^{\ell_2}, \quad (5)$$

where $\mathbf{u}' = (\text{eq}(\mathbf{b}, \mathbf{r}))_{\mathbf{b} \in \{0, 1\}^{\log \ell_1}}$ for some $\mathbf{r} \in \mathbb{F}_{2^n}^{\log \ell_1}$, and for some $\kappa_j \in \mathbb{F}_{2^n}$.

4. Note that (5) is a collection of standard claims about the MLE evaluation of a vector with entries in $\mathbb{F}_{2^n}$, “committed” through an encoding over $\mathbb{F}_{2^n}$. Accordingly, prover and verifier use an IOPP to prove such claims. [Albert: To keep in mind: ring-switch for arbitrary inner products could be expensive](#)

Example 4.2. Say we have a vector $\mathbf{f} \in \{0, 1\}^\ell \subseteq \mathbb{Z}^\ell$ and we wish to prove MLE evaluation claims of an extended vector $\mathbf{f}' \in \{0, 1\}^{2 \cdot \ell}$ where

$$\mathbf{f}'_i = \begin{cases} \mathbf{f}_i & \text{if } i \in [\ell], \\ \mathbf{f}_{i-\ell} \text{ XOR } \mathbf{f}_{i-\ell+1} & \text{if } \ell + 1 \leq i \leq 2 \cdot \ell - 1, \\ \mathbf{f}_\ell & \text{if } i = 2 \cdot \ell. \end{cases}$$

In words, $\mathbf{f}'$ is the result of concatenating $\mathbf{f}$ and the vector obtained by “xor-ing” together every two consecutive elements in $\mathbf{f}$ (and keeping the last entry as $\mathbf{f}_\ell$, since $\mathbf{f}_\ell \text{ XOR } \mathbf{f}_{\ell+1}$ is not defined). This example will come up in our SHA-256 case-study.

We have

$$\mathbf{f}' = \pi_2^{-1}(M \cdot \pi_2(\mathbf{f})) \in \{0, 1\}^{2\ell}$$

where

$$M = \begin{pmatrix} & & \text{Id}_{\ell \times \ell} & & & \\ \hline 1 & 1 & 0 & 0 & \dots & 0 \\ 0 & 1 & 1 & 0 & \dots & 0 \\ \vdots & & \ddots & & & \vdots \\ 0 & \dots & 0 & 0 & 1 & 1 \\ 0 & \dots & 0 & 0 & 0 & 1 \end{pmatrix}$$

if we do not wish to leverage any tensor decomposition structure, i.e. if we take $\ell_1 = \ell$ and $\ell_2 = 1$, then we can apply [Construction 4.1](#) using this matrix M as part of the input.

At the beginning of Step 3 of [Construction 4.1](#), prover and verifier obtain the claim

$$((\mathbf{u}, \eta), \text{Enc}_C(\text{Pack}_n(\pi_2(\mathbf{f}))), \pi_2(\mathbf{f})) \in \mathcal{R}_{C_1, \mathbb{F}_{2^n}, \text{Pack}_n}^{\text{lin}},$$

for

$$\mathbf{u} = (\text{mle}[M_{*t}](\mathbf{r}))_{t \in [\ell]} \in \mathbb{F}_{2^n}^\ell,$$

where M_{*t} denotes the t -th column of M . Then, prover and verifier use the Galois ring-switch [Construction A.1](#) to reduce the above claim to standard MLE evaluation claims of $\text{Pack}_n(\pi_2(\mathbf{f}_{*j}))$. The cost of doing so for the vector $\mathbf{u}$ of linear coefficients is [Albert: discuss.](#)

Next we address the case when we use a non-trivial tensor decomposition structure $\ell = \ell_1 \cdot \ell_2$ with $\ell_1 \neq 1 \neq \ell_2$. [Albert: TODO](#)

Theorem 4.3. *Albert: Completeness and RBR knowledge soundness of Construction 3.1*

Proof. □

Remark 4.4 (Batching families of virtual claims: case-collapsed weight functions). [Construction 4.1](#) proves a *single* lin-virtual claim, and its Π_{red} instance runs over the ℓ' *virtual* entries $M \cdot \pi_2(\mathbf{f})$; k claims cost k such instances. When the k claimed combinations are supported on a *small* shared set of $j \leq 4$ committed columns, and all claims share the second tensor component of their weight vectors (in the MLE case: the same “column” sub-point $\mathbf{r}^{(2)}$, with the row sub-points free to differ per claim), there is an alternative that runs *one* Π_{red} instance over the *committed* entries only. After the statement is fixed, batching coefficients $\gamma_1, \dots, \gamma_k \leftarrow \mathbb{F}_q$ are drawn and pushed *into the weight function*, reducing modulo q *before* the lift into the exponents of g : replacing a public weight function by its representative in $[0..q-1]$ is invisible to the mod- q statement, so the field-sized randomness costs *zero* exponent budget — the door that integer-coefficient random linear combinations cannot use, since integer coefficients enlarge the lifted weights the exponent encoding must carry. The combined weight at a position then depends on the committed data only through the j -bit *case vector* of that position, so the leaves of Π_{red} become a public 2^j -case select, multilinear in the j committed bits via a characteristic-2 subset zeta transform; the exit sumcheck splits into $2^j - 1$ channels, and the channels of degree ≥ 2 — precisely the AND/carry content that the identities at the start of this section show to be genuinely new information — are discharged by one degree- $(j+1)$ **eq**-sumcheck over the cube in $\mathbb{F}_{2^n}$, exiting at committed openings. The combination costs soundness $1/q$ (q prime; one linear form carries all k discrepancies at once).

If moreover *all* k points coincide — the deployment norm, e.g. several combinations of the same columns opened at one PCS query point — the case table is rank-one, $W_b(m) = w_b \cdot \Gamma(m) \bmod q$ with $\Gamma(m) = \sum_i \gamma_i L_i(m)$ taking 2^j values in total; two claims with the same combination are the *same* claim, so the family is canonical with $k \leq 2^j - 1$, the maximal family being the full XOR-closure of the j columns; and the Fiat–Shamir statement absorbs a single weight vector instead of k . Measured in the reference implementation (Apple M4, 2^{26} committed bits): the maximal $j = 4$ family ($k = 15$) proves at $0.36\times$ one claim *per claim*, about $2.2\times$ faster than the same statement through the derived-row path with 77% smaller proofs — and at that width the monomial discharge, not the product argument, dominates the prover. The derived-row path of [Construction 4.1](#) remains the right tool for lone claims and for wide-support rows (the case tables scale as 2^j). [Albert: New \(2026-07-27\)](#), distilled from the implementation repository: the construction, its soundness proof, the vanishing-channel completeness refinement, and the full measurement tables live in [docs/rlc-family-note/](#) and the README’s dated notes. Integrate properly or trim as we see fit.

5 R1CS constraints over polynomial rings with ideal membership

6 Case-study: SHA-256 hashing

Albert: Maybe Keccak better?

6.1 Arithmetization over $\mathbb{Z}[X]$

6.1.1 SHA-256 compression overview

We begin with a brief overview of SHA-256 to establish notation, then formalize the relation we seek to arithmetize.

For the purposes of SHA-256, a *word* is a ≤ 32 string of bits, which we represent as a binary integer polynomial, i.e. as an element of $\{0, 1\}^{<32}[X]$. The SHA-256 compression function receives as input a 512-bit string, represented as sixteen *message words* $(M_t)_{t \in [16]} \in \{0, 1\}^{<32}[X]^{16}$. The algorithm also uses 64 fixed *round constants* $(K_t)_{t \in [64]} \in \{0, 1\}^{<32}[X]^{64}$.

We let

$$\begin{aligned} \psi_2 : \mathbb{Z}[X] &\rightarrow \mathbb{Z} \\ a(X) &\mapsto a(2) \end{aligned} \tag{6}$$

be the ring homomorphism sending integer polynomials to their evaluation at $X = 2$. Notice that, given $a \in \{0, 1\}^{<32}[X]$, the integer $\psi_2(a) \in \mathbb{Z}$ is the integer whose bit-representation coincides with the coefficients of a .

SHA-256 is built from the following six auxiliary functions. For $x, y, z \in \{0, 1\}^{<32}[X]$, define:

$$\begin{aligned} \Sigma_0(x) &:= \text{ROTR}^2(x) \text{ XOR } \text{ROTR}^{13}(x) \text{ XOR } \text{ROTR}^{22}(x) && \in \{0, 1\}^{<32}[X] \\ \Sigma_1(x) &:= \text{ROTR}^6(x) \text{ XOR } \text{ROTR}^{11}(x) \text{ XOR } \text{ROTR}^{25}(x) && \in \{0, 1\}^{<32}[X] \\ \text{Ch}(x, y, z) &:= (x \text{ AND } y) \text{ XOR } (\neg x \text{ AND } z) && \in \{0, 1\}^{<32}[X] \\ \text{Maj}(x, y, z) &:= (x \text{ AND } y) \text{ XOR } (x \text{ AND } z) \text{ XOR } (y \text{ AND } z) && \in \{0, 1\}^{<32}[X] \\ \sigma_0(x) &:= \text{ROTR}^7(x) \text{ XOR } \text{ROTR}^{18}(x) \text{ XOR } \text{SHR}^3(x) && \in \{0, 1\}^{<32}[X] \\ \sigma_1(x) &:= \text{ROTR}^{17}(x) \text{ XOR } \text{ROTR}^{19}(x) \text{ XOR } \text{SHR}^{10}(x) && \in \{0, 1\}^{<32}[X] \end{aligned} \tag{7}$$

Albert: todo: define the bit operations

Message schedule. At the beginning of the SHA-256 compression function, the sixteen input message words $(M_t)_{t \in [16]}$ are expanded into a so-called *message schedule*, which con-

sists of 64 integers of 32 bits $(W_t)_{t \in [64]} \in \mathbb{Z}^{64}$ defined as follows:

$$W_t := \begin{cases} \psi_2(M_t) & t \in [16], \\ \psi_2(W_{t-16}) + \psi_2(\sigma_0(W_{t-15})) + \psi_2(W_{t-7}) + \psi_2(\sigma_1(W_{t-2})) \pmod{2^{32}} & t \in \{17, \dots, 64\}. \end{cases} \quad (8)$$

Compression loop. After the message schedule is computed, SHA-256 performs the so-called *compression loop*. Throughout it, the function maintains an 8-word internal state, conventionally labeled as

$$(a, b, c, d, e, f, g, h) \in \{0, 1\}^{<32}[X]^8.$$

The compression loop iterates 64 rounds. At each round $t \in [64]$, the state entering that round is denoted $(a_t, b_t, c_t, d_t, e_t, f_t, g_t, h_t) \in \{0, 1\}^{<32}[X]^8$. The per-round update is given by, for $t \in [64]$,

$$\begin{aligned} T_{1,t} &:= \psi_2(h_t) + \psi_2(\Sigma_1(e_t)) + \psi_2(\text{Ch}(e_t, f_t, g_t)) + \psi_2(K_t) + W_t \pmod{2^{32}} && \in \mathbb{Z}, \\ T_{2,t} &:= \psi_2(\Sigma_0(a_t)) + \psi_2(\text{Maj}(a_t, b_t, c_t)) \pmod{2^{32}} && \in \mathbb{Z}, \\ a_{t+1} &:= \psi_2^{-1}(T_{1,t} + T_{2,t} \pmod{2^{32}}) && \in \{0, 1\}^{<32}[X], \\ e_{t+1} &:= \psi_2^{-1}(\psi_2(d_t) + \psi_2(T_{1,t}) \pmod{2^{32}}) && \in \{0, 1\}^{<32}[X], \\ (b_{t+1}, c_{t+1}, d_{t+1}, f_{t+1}, g_{t+1}, h_{t+1}) &:= (a_t, b_t, c_t, e_t, f_t, g_t) && \in \{0, 1\}^{<32}[X]^6. \end{aligned}$$

The full SHA-256 compression function adds the initial state to the terminal state word-wise modulo 2^{32} (the so-called *feed-forward step*); we omit this step for brevity [Albert: Let's not do that](#). For our purposes, the output of the compression function is the final state $(a_{65}, \dots, h_{65})$.

6.1.2 Our arithmetization

6.2 End-to-end IOP

6.3 Experiments

[Albert: GHASH or b127?](#)

[Albert: Comparison with Flock on SHA256: we commit to 8x less bits. This leads to more than 2x memory reduction. 80% of prover costs are in the grand product GKR. Hence setting rate=1/4 or smaller has little impact on performance. This leads to smaller proofs. Prover time overall seems to be about 1.5-2.5x worse for us. I also switched to a better config of Ligerito \(called k=4 in impl\), which reduces proof size by almost 2x, but increases open time, which for us adds a negligible overhead, while in Flock it adds more overhead](#)

Optimization ideas

- b127 instead of GHASH
- Lookup table (algorithm-wise, not SNARK-wise) for first levels of GKR
- If we have an MLE claim of a vector and its shift by one position, can we reuse something for the latter claim from the computations made when proving the former?

7 Case-study: Keccak-f[1600] hashing

7.1 Arithmetization over $\mathbb{Z}[X]$

7.1.1 Keccak-f[1600] overview

We begin with a brief overview of the permutation Keccak- f [1600] (called Keccak- p [1600, 24] in the SHA-3 standard [Nat15]) to establish notation, then formalize the relation we seek to arithmetize.

For the purposes of Keccak- f [1600], a *lane* is a ≤ 64 string of bits, which we represent as a binary integer polynomial, i.e. as an element of $\{0, 1\}^{<64}[X]$. The *state* of the permutation is a 5×5 array of lanes

$$A = (A_{x,y})_{x,y \in \{0,\dots,4\}} \in (\{0, 1\}^{<64}[X])^{5 \times 5},$$

comprising 1600 bits in total; the coefficient of X^z in $A_{x,y}$ is the state bit with coordinates (x, y, z) in the standard specification. Throughout this section, all lane indices are understood modulo 5. The algorithm also uses 24 fixed *round constants* $(RC_t)_{t \in [24]} \in \{0, 1\}^{<64}[X]^{24}$, whose values are specified in [Nat15]; each RC_t has nonzero coefficients only in degrees $2^j - 1$ for $j \in \{0, \dots, 6\}$, so it is supported on at most 7 of the 64 bit positions.

The bitwise operations XOR, AND, and NOT ($\neg$) act coefficient-wise on binary integer polynomials. Besides these, Keccak uses the cyclic left-rotation of lanes, which in our representation is multiplication by a monomial: for $a = \sum_{i=0}^{63} a_i X^i \in \{0, 1\}^{<64}[X]$ and $r \geq 0$, we define

$$\text{ROTL}^r(a) := X^r \cdot a \bmod (X^{64} - 1) = \sum_{i=0}^{63} a_{(i-r) \bmod 64} \cdot X^i \in \{0, 1\}^{<64}[X].$$

The reduction modulo $X^{64} - 1$ sends each monomial $a_i X^{i+r}$ to $a_i X^{(i+r) \bmod 64}$, and no two monomials collide; thus ROTL^r merely permutes coefficients—no coefficient arithmetic takes place—and in particular it maps binary integer polynomials to binary integer polynomials.

Step mappings. Given a state A , define its *column parities* $C_0, \dots, C_4$ by

$$C_x := A_{x,0} \text{ XOR } A_{x,1} \text{ XOR } A_{x,2} \text{ XOR } A_{x,3} \text{ XOR } A_{x,4} \in \{0,1\}^{<64}[X].$$

Keccak- $f[1600]$ is built from five *step mappings* $\theta, \rho, \pi, \chi, \iota_t$, each sending states to states, defined lane-wise as follows: for $x, y \in \{0, \dots, 4\}$,

$$\begin{aligned} \theta(A)_{x,y} &:= A_{x,y} \text{ XOR } C_{x-1} \text{ XOR } \text{ROTL}^1(C_{x+1}), \\ \rho(A)_{x,y} &:= \text{ROTL}^{r_{x,y}}(A_{x,y}), \\ \pi(A)_{x,y} &:= A_{x+3y, x}, \\ \chi(A)_{x,y} &:= A_{x,y} \text{ XOR } (\neg A_{x+1,y} \text{ AND } A_{x+2,y}), \\ \iota_t(A)_{x,y} &:= \begin{cases} A_{0,0} \text{ XOR } RC_t & \text{if } (x, y) = (0, 0), \\ A_{x,y} & \text{otherwise,} \end{cases} \end{aligned} \tag{9}$$

where the rotation offsets $r_{x,y} \in \{0, \dots, 63\}$ used by ρ are the fixed constants

$r_{x,y}$	$x = 0$	$x = 1$	$x = 2$	$x = 3$	$x = 4$
$y = 0$	0	1	62	28	27
$y = 1$	36	44	6	55	20
$y = 2$	3	10	43	25	39
$y = 3$	41	45	15	21	8
$y = 4$	18	2	61	56	14

(10)

In words: θ XORs into each lane the parity of the column to its left and the (once-rotated) parity of the column to its right; ρ rotates each lane by a fixed offset; π permutes the 25 lane positions; χ XORs into each lane a bitwise function of the next two lanes in its row, and is the only nonlinear step; and ι_t XORs the round constant RC_t into the lane $A_{0,0}$.

The permutation. The *round function* at round $t \in [24]$ is the composition

$$R_t := \iota_t \circ \chi \circ \pi \circ \rho \circ \theta,$$

and Keccak- $f[1600]$ iterates it 24 times: on input a state $A^{(1)}$, the states

$$A^{(t+1)} := R_t(A^{(t)}), \quad t \in [24], \tag{11}$$

are computed in order, and the output of the permutation is the final state $A^{(25)}$.

Sponge hashing. The hash functions of the SHA-3 family, as well as the Keccak-256 function used in Ethereum (the two differ only in the message padding), are obtained from Keccak- $f[1600]$ via the sponge construction: the state is initialized to zero, the padded

message is *absorbed* r bits at a time—each absorption XORs an r -bit message block into a fixed r -bit portion of the state and then applies Keccak- $f[1600]$ —and the digest is finally read off the state. For a 256-bit digest one takes rate $r = 1088$ (and capacity $c = 1600 - r = 512$), so hashing costs one invocation of the permutation per 1088 message bits, and the permutation is the only cryptographic component of the hash. For our purposes, the unit of work is a single invocation of the permutation: the relation we arithmetize is the conjunction of the 24 round relations (11).

Structural properties. The steps θ , ρ , π are $\mathbb{F}_2$ -linear on the 1600 state bits, and ι_t is the XOR of a public constant; hence all the nonlinearity of a round resides in χ , which has algebraic degree 2. Writing $\lambda := \pi \circ \rho \circ \theta$ for the linear layer, each bit of $\lambda(A)$ is the XOR of exactly 11 bits of A (θ XORs each state bit with the ten bits of its two neighbouring column parities, while ρ and π only permute bit positions), and each of the 1600 bits of $\chi(A)$ is computed from A with exactly one bitwise AND, of the form $\neg b \text{ AND } c$. A round thus decomposes as one (11-sparse) $\mathbb{F}_2$ -linear layer, followed by one layer of 1600 AND gates, followed by the XOR of a public constant—for a total of $24 \cdot 1600 = 38400$ AND gates per permutation. Moreover, in contrast with SHA-256, Keccak performs no integer additions: the evaluation-at-2 homomorphism ψ_2 used for SHA-256 plays no role, and no carry bits ever arise. Looking ahead, this makes Keccak- $f[1600]$ a particularly good fit for our framework: every $\mathbb{F}_2$ -linear layer can be virtualized in the sense of Section 4, i.e. absorbed into the matrix M and never committed, while each AND gate is discharged by a linear identity analogous to those of Section 4, as we describe next.

7.1.2 Our arithmetization

Committed witness. For each invocation of the permutation we commit to the states at the 25 round boundaries: the witness is the bit-vector

$$\mathbf{f} = \left(A^{(t)} \right)_{t \in [25]} \in \{0, 1\}^{40000},$$

where each state contributes the $25 \cdot 64 = 1600$ coefficients of its 25 lanes, so that $40000 = 25 \cdot 1600$. In addition, we adjoin to $\mathbf{f}$ a single auxiliary bit $\mathbf{f}_{\text{one}}$, forced to equal 1 by the public linear claim $\pi_q(\mathbf{f}_{\text{one}}) = 1$; rows of M whose supports include this bit realize XORs with public constants (such as the round constants) while remaining $\mathbb{F}_2$ -linear in the committed bits. No further witness material is needed: in particular no carry bits, and all intermediate values of the rounds remain virtual in the sense of Section 4.

The round relations as linear constraints. Fix a round $t \in [24]$ and let $B^{(t)} := \lambda(A^{(t)}) = \pi(\rho(\theta(A^{(t)})))$ be the state entering the nonlinear step; each bit of $B^{(t)}$ is the XOR of exactly 11 committed bits. Lane-wise, the round relation $A^{(t+1)} = \iota_t(\chi(B^{(t)}))$

reads

$$A_{x,y}^{(t+1)} = B_{x,y}^{(t)} \text{ XOR } \left(\neg B_{x+1,y}^{(t)} \text{ AND } B_{x+2,y}^{(t)} \right) \text{ XOR } \varepsilon_{x,y}, \quad x, y \in \{0, \dots, 4\},$$

where $\varepsilon_{0,0} = RC_t$ and $\varepsilon_{x,y} = 0$ for $(x, y) \neq (0, 0)$. Fix additionally a bit position z , and write $a, b, c, a', rc \in \{0, 1\}$ for the coefficients of X^z in $B_{x,y}^{(t)}$, $B_{x+1,y}^{(t)}$, $B_{x+2,y}^{(t)}$, $A_{x,y}^{(t+1)}$ and $\varepsilon_{x,y}$, respectively (so rc is public). The round relations are then the conjunction, over the $24 \cdot 1600 = 38400$ choices of (t, x, y, z) , of the bit relations

$$a' = a \text{ XOR } (\neg b \text{ AND } c) \text{ XOR } rc. \quad (12)$$

Analogously to the identities for AND and Majority from [Section 4](#), for integer bits $b, c \in \{0, 1\} \subseteq \mathbb{Z}$ we have

$$\pi_q(\neg b \text{ AND } c) = 2^{-1} \cdot \pi_q(c - b + \pi_2^{-1}(\pi_2(b) + \pi_2(c))), \quad (13)$$

where, as in [Section 4](#), 2^{-1} denotes the inverse of 2 modulo q . Since (12) is equivalent to $\neg b \text{ AND } c = a' \text{ XOR } a \text{ XOR } rc$ (XOR both sides with $a \text{ XOR } rc$), identity (13) turns it into the $\mathbb{F}_q$ -linear constraint

$$2 \cdot \pi_q(a' \text{ XOR } a \text{ XOR } rc) + \pi_q(b) - \pi_q(c) - \pi_q(b \text{ XOR } c) = 0. \quad (14)$$

Conversely, since all quantities involved are bits, the left-hand side of (14), read over $\mathbb{Z}$ before reduction, lies in $\{-2, \dots, 2\}$; hence for any odd q (or $q = 0$) constraint (14) holds if and only if the bit relation (12) does.

Every quantity to which π_q is applied in (14) is (the lift to $\mathbb{Z}$ of) an $\mathbb{F}_2$ -linear combination of committed bits, i.e. an entry of $\pi_2^{-1}(M \cdot \pi_2(\mathbf{f}))$ for a suitable public matrix M . Concretely, the constraints reference three families of virtual entries, indexed by the 38400 tuples (t, x, y, z) :

- the bits of $B^{(t)}$ (each the XOR of 11 committed bits),
- the bits of $B_{x+1,y}^{(t)} \text{ XOR } B_{x+2,y}^{(t)}$ (each the XOR of at most 22 committed bits), and
- the bits of $A_{x,y}^{(t+1)} \text{ XOR } B_{x,y}^{(t)} \text{ XOR } \varepsilon_{x,y}$ (each the XOR of 12 committed bits, plus $\mathbf{f}_{\text{one}}$ when the corresponding round-constant bit is 1),

so that M has (at most) $3 \cdot 38400 = 115200$ such rows per invocation, each with at most 22 nonzero entries, plus identity rows exposing the committed bits that appear in public input/output claims. The constraint family (14) is a public $\mathbb{F}_q$ -linear map, with coefficients in $\{2, \pm 1\}$, applied to the virtual vector $\pi_q(\pi_2^{-1}(M \cdot \pi_2(\mathbf{f})))$ and required to vanish identically; a random $\mathbb{F}_q$ -linear combination of these constraints, together with the public claims ($\pi_q(\mathbf{f}_{\text{one}}) = 1$ and the exposure of the input and output states), yields claims of the form $\langle \pi_q(\pi_2^{-1}(M \cdot \pi_2(\mathbf{f}))), \mathbf{v} \rangle = \mu$, i.e. instances of $\mathcal{R}_{C, \mathbb{F}_{2^n}, \mathbb{F}_q, \text{Pack}_n}^{\text{lin-virtual}}$, which we prove with [Construction 4.1](#).

Batching. When batching ℓ_2 invocations of the permutation (e.g. the permutation calls of a long absorbing phase, or independent hashes), the j -th invocation occupies the sub-vector $\mathbf{f}_{*j}$; since Keccak is unkeyed and every invocation uses the same round structure, the same block M_0 works for every j , and the matrix is exactly of the block-diagonal form $M = \text{Id} \otimes M_0$ assumed in [Construction 4.1](#).

Albert: TODO: (i) spell out how the 38400 constraints (14), together with the public claims, are batched into claims of the form fed to [Construction 4.1](#); (ii) chaining consecutive sponge blocks ($A^{(1)}$ of block $j+1 = A^{(25)}$ of block j XOR the j -th message block) is $\mathbb{F}_2$ -linear but crosses columns, so it does not fit the block-diagonal shape $M = \text{Id} \otimes M_0$ as-is—discuss; (iii) cost of evaluating $\text{mle}[M_{0*t}](\mathbf{r})$ for this M_0 , exploiting its uniformity across (t, x, y, z) .

7.2 End-to-end IOP

7.3 Experiments

A Ring switching via Galois orbits

Albert: Rewriting the previous appendix for arbitrary inner products

Construction A.1. [Galois ring-switch: an IOR from $\mathcal{R}_{\mathcal{C}, \mathbb{F}_{2^n}, \text{Pack}_n}^{\text{lin}}$ to $\mathcal{R}_{\mathcal{C}_1, \mathbb{F}_{2^n}, \text{id}}^{\text{mle}}$] Albert: TODO: formulate as a reduction from $\mathcal{R}_{\mathcal{C}, \mathbb{F}_{2^n}, \text{Pack}_n}^{\text{lin}}$ to $\mathcal{R}_{\mathcal{C}_1, \mathbb{F}_{2^n}, \text{id}}^{\text{mle}}$.

Let $b_1, \dots, b_m \in \mathbb{F}_2^{2^n}$. Using an $\mathbb{F}_2$ -basis $\beta_1, \dots, \beta_m$ of $\mathbb{F}_{2^m}$ we pack m inner products

$$\langle b_i, \mathbf{v} \rangle = \sum_{j \in \{0,1\}^n} b_{i,j} \cdot \mathbf{v}_j, \quad i = 1, \dots, m,$$

coefficient-wise into a single inner product

$$\begin{aligned} b &:= b_1 \cdot \beta_1 + \dots b_m \cdot \beta_m \in \mathbb{F}_{2^m}^{2^n} \\ \langle b, \mathbf{v} \rangle &= \langle b_1, \mathbf{v} \rangle \cdot \beta_1 + \dots + \langle b_m, \mathbf{v} \rangle \cdot \beta_m \\ &= \sum_{j \in \{0,1\}^n} (b_{1,j} \cdot \beta_1 + \dots + b_{m,j} \cdot \beta_m) \cdot \mathbf{v}_j. \end{aligned}$$

The Frobenius automorphism $\phi(z) = z^2$ keeps $\mathbb{F}_2 \subset \mathbb{F}_{2^m}$ fixed, and has no other fixed points. We extract the value of $\langle b_i, \mathbf{v} \rangle$ from the values of $\langle b, \mathbf{v} \rangle$ along the Galois orbit

$$\mathbf{v}, \mathbf{v}^2, \mathbf{v}^{2^2}, \dots, \mathbf{v}^{2^{m-1}},$$

as follows, where by $\mathbf{v}^t$ we mean that each component of $\mathbf{v}$ is raised to the power t . The

Moore matrix

$$M = \begin{pmatrix} \beta_1 & \beta_2 & \dots & \beta_m \\ \beta_1^2 & \beta_2^2 & \dots & \beta_m^2 \\ \beta_1^{2^2} & \beta_2^{2^2} & \dots & \beta_m^{2^2} \\ \vdots & & & \\ \beta_1^{2^{m-1}} & \beta_2^{2^{m-1}} & \dots & \beta_m^{2^{m-1}} \end{pmatrix}$$

consists of the Galois orbits of the basis vectors, and is invertible. (See below.) Using that the component polynomials are over $\mathbb{F}_2$, we see that

$$\phi^k \left(\langle b, \phi^{-k}(\mathbf{v}) \rangle \right) = \langle b_1, \mathbf{v} \rangle \cdot \phi^k \beta_1 + \dots + \langle b_m, \mathbf{v} \rangle \cdot \phi^k \beta_m,$$

for any k . To this end, observe that

$$M \cdot \begin{pmatrix} \langle b_1, \mathbf{v} \rangle \\ \langle b_2, \mathbf{v} \rangle \\ \langle b_3, \mathbf{v} \rangle \\ \vdots \\ \langle b_m, \mathbf{v} \rangle \end{pmatrix} = \begin{pmatrix} \langle b, \mathbf{v} \rangle \\ \phi \left(\langle b, \phi^{-1}(\mathbf{v}) \rangle \right) \\ \phi^2 \left(\langle b, \phi^{-2}(\mathbf{v}) \rangle \right) \\ \vdots \\ \phi^{m-1} \left(\langle b, \phi^{-(m-1)}(\mathbf{v}) \rangle \right) \end{pmatrix}.$$

Multiplying both sides by M^{-1} we obtain an expression of the values $\langle b_i, \mathbf{v} \rangle$ as a linear expression of the values $\phi^k \left(\langle b, \phi^{-k}(\mathbf{v}) \rangle \right)$. **Ulrich:** Invertibility of M follows from that of M^t : any linear functional $(c_1, \dots, c_m)$ in the kernel of M^t gives rise to the linearizable polynomial

$$p(X) = \sum_{i=0}^{m-1} c_i \cdot X^{2^i}$$

which has $\beta_1, \dots, \beta_m$, and hence the whole space F as roots. By dimension, $p(X)$ must be trivial, meaning that all its coefficients are zero.

B [Old appendix by Ulrich] Galois FRI-Binius

Ulrich: Just notes, not sure if it worth to keep it in final doc **Albert:** Nice, thanks!

Packed commitment scheme for $\mathbb{F}_{2^m}$.

For simplicity we assume packing across polynomials. (The single poly case goes likewise.) Using an $\mathbb{F}_2$ -basis $\beta_1, \dots, \beta_m$ of $\mathbb{F}_{2^m}$ we pack m polys of 2^n coefficients

$$b_i(X) = \sum_{j=0}^{2^n-1} b_{i,j} \cdot X^j \in \mathbb{F}_2[X]^{<2^n}, \quad i = 1, \dots, m,$$

coefficient-wise into a single poly

$$\begin{aligned} b(X) &= b_1(X) \cdot \beta_1 + \dots + b_m(X) \cdot \beta_m \\ &= \sum_{j=0}^{2^n} (b_{1,j} \cdot \beta_1 + \dots + b_{m,j} \cdot \beta_m) \cdot X^j \in \mathbb{F}_{2^m}[X]^{<2^n}. \end{aligned}$$

The Frobenius automorphism $\phi(z) = z^2$ keeps $\mathbb{F}_2 \subset F$ fixed, and has no other fixed points. For any point $z \in \mathbb{F}$, we extract the value of $b_i(z)$ from the values of the packed poly $b(X)$ along the Galois orbit

$$z, z^2, z^{2^2}, \dots, z^{2^{m-1}}$$

as follows. The Moore matrix

$$M = \begin{pmatrix} \beta_1 & \beta_2 & \dots & \beta_m \\ \beta_1^2 & \beta_2^2 & \dots & \beta_m^2 \\ \beta_1^{2^2} & \beta_2^{2^2} & \dots & \beta_m^{2^2} \\ \vdots & & & \\ \beta_1^{2^{m-1}} & \beta_2^{2^{m-1}} & \dots & \beta_m^{2^{m-1}} \end{pmatrix}$$

consists of the Galois orbits of the basis vectors, and is invertible. (See below.) Using that the component polynomials are over $\mathbb{F}_2$, we see that

$$\phi^k \left(b(\phi^{-k} z) \right) = b_1(z) \cdot \phi^k \beta_1 + \dots + b_m(z) \cdot \phi^k \beta_m,$$

for any k . To extract the $b_1(z)$ term, we look for weights $c_1, \dots, c_m \in F$ which annihilate the orbits of the other $\beta_2, \dots, \beta_m$, concretely

$$\sum_{k=0}^{m-1} c_k \cdot \phi^k \beta_1 = 1, \quad \sum_{k=0}^{m-1} c_k \cdot \phi^k \beta_j = 0 \quad j = 2, \dots, m.$$

This is exactly the first row vector of M^{-1} . In general,

$$\begin{pmatrix} b_1(z) \\ b_2(z) \\ b_3(z) \\ \vdots \\ b_m(z) \end{pmatrix} = M^{-1} \cdot \begin{pmatrix} v_0 \\ \phi v_1 \\ \phi^2 v_2 \\ \vdots \\ \phi^{m-1} v_{m-1} \end{pmatrix},$$

where $v_k = b(\phi^{-k} z)$. **Ulrich:** Invertibility of M follows from that of M^t : any linear functional $(c_1, \dots, c_m)$ in the kernel of M^t gives rise to the linearizable polynomial

$$p(X) = \sum_{i=0}^{m-1} c_i \cdot X^{2^i}$$

which has $\beta_1, \dots, \beta_m$, and hence the whole space F as roots. By dimension, $p(X)$ must be trivial, meaning that all its coefficients are zero.

C Bilinear Embeddings

Given a field $\mathbb{F}$ and a degree- d extension $\mathbb{E}$. Our goal is to commit to a vector $\mathbf{w} \in \mathbb{F}^{nd}$ so that we can later use it to prove an inner product claim of the form $\langle \mathbf{w}, \mathbf{a} \rangle_{\mathbb{F}} = h$ for some $\mathbf{a} \in \mathbb{F}^{nd}$ and $h \in \mathbb{F}$. However, we only have an efficient inner product commitment scheme to commit to vectors $\tilde{\mathbf{w}} \in \mathbb{E}^n$ and open as $\langle \tilde{\mathbf{w}}, \tilde{\mathbf{a}} \rangle_{\mathbb{E}} = \tilde{h}$ for some $\tilde{\mathbf{a}} \in \mathbb{E}^n$ and $\tilde{h} \in \mathbb{E}$.

To do this, we seek a triple of maps (W, A, H) that satisfy:

$$\langle \mathbf{w}, \mathbf{a} \rangle_{\mathbb{F}} = H(\langle W(\mathbf{w}), A(\mathbf{a}) \rangle_{\mathbb{E}}).$$

C.1 Background

In the following let $\mathbb{E}/\mathbb{F}$ be a finite field extension of degree d . If we disregard multiplication in $\mathbb{E}$, then $\mathbb{E}$ is a d -dimensional vector space over $\mathbb{F}$, hence isomorphic to $\mathbb{F}^d$. Let $\mathbf{e}_1, \dots, \mathbf{e}_d$ denote the standard basis of $\mathbb{F}^d$. Recall that a function $\mu : \mathbb{E} \times \mathbb{E} \rightarrow \mathbb{F}$ is a *bilinear form* if it is linear in each argument separately. It is *symmetric* if $\mu(x, y) = \mu(y, x)$ for all $x, y \in \mathbb{E}$. For each $x \in \mathbb{E}$, the partial application in the first argument defines a linear form

$$\mu_x : \mathbb{E} \rightarrow \mathbb{F} \quad y \mapsto \mu(x, y).$$

Thus $\mu_x \in \mathbb{E}^*$ where $\mathbb{E}^* = \text{Hom}_{\mathbb{F}}(\mathbb{E}, \mathbb{F})$ denotes the dual vector space. The bilinear form therefore induces an $\mathbb{F}$ -linear map

$$\Phi_{\mu} : \mathbb{E} \rightarrow \mathbb{E}^* \quad x \mapsto \mu_x.$$

We call μ *nondegenerate*, or equivalently a *perfect pairing*, if Φ_{μ} is an isomorphism. Or equivalently for finite spaces, [Albert: add a comment that injectivity implies surjectivity when the vector spaces have finite dimension?](#)

$$\mu(x, y) = 0 \text{ for all } y \in \mathbb{E} \implies x = 0.$$

Since μ is a perfect pairing, every basis $(\beta_1, \dots, \beta_d)$ of $\mathbb{E}$ has a unique dual basis $(\beta_1^{\vee}, \dots, \beta_d^{\vee})$ satisfying $\mu(\beta_i, \beta_j^{\vee}) = \delta_{ij}$.

C.2 Structure

We are now equipped to prove a theorem on the structure of embeddings. Under the assumption that the output map is $\mathbb{F}$ -linear, we show that for $n = 1$ the space of possible embeddings is very structured.

Theorem C.1 (Structure of bilinear embeddings). *Let $W, A : \mathbb{F}^d \rightarrow \mathbb{E}, H : \mathbb{E} \rightarrow \mathbb{F}$ be maps such that for all $\mathbf{w}, \mathbf{a} \in \mathbb{F}^d$,*

$$\langle \mathbf{w}, \mathbf{a} \rangle_{\mathbb{F}} = H(W(\mathbf{w})A(\mathbf{a})).$$

Suppose that H is $\mathbb{F}$ -linear. Then:

1. W and A are bijective $\mathbb{F}$ -linear embeddings,
2. $\mu_H(x, y) = H(xy)$ is a perfect pairing,
3. $(W(\mathbf{e}_1), \dots, W(\mathbf{e}_d))$ and $(A(\mathbf{e}_1), \dots, A(\mathbf{e}_d))$ are dual bases with respect to μ_H .

Conversely, if W and A , and H are $\mathbb{F}$ -linear, μ_H is a perfect pairing and the images of the standard basis under W and A are dual bases with respect to μ_H , then the identity

$$\langle \mathbf{w}, \mathbf{a} \rangle_{\mathbb{F}} = H(W(\mathbf{w})A(\mathbf{a})).$$

holds for all $\mathbf{w}, \mathbf{a} \in \mathbb{F}^d$.

Proof of the forward direction. First, observe that $\mu_H(W(\mathbf{e}_i), A(\mathbf{e}_j)) = \langle \mathbf{e}_i, \mathbf{e}_j \rangle_{\mathbb{F}} = \delta_{ij}$. Taking $i = j$ shows H is nonzero. Since H is $\mathbb{F}$ -linear, μ_H is a symmetric bilinear form, we show that it is nondegenerate. Let $x \in \mathbb{E}$ be nonzero. Since multiplication by x is a bijection of $\mathbb{E}$ and H is nonzero, there exists $y \in \mathbb{E}$ such that $\mu_H(x, y) = H(xy) \neq 0$. Thus μ_H is nondegenerate, and hence a perfect pairing.

Next, we show that W and A are bijective. Suppose $W(\mathbf{w}_1) = W(\mathbf{w}_2)$. Then for every $\mathbf{a} \in \mathbb{F}^d$,

$$\langle \mathbf{w}_1 - \mathbf{w}_2, \mathbf{a} \rangle_{\mathbb{F}} = \mu_H(W(\mathbf{w}_1) - W(\mathbf{w}_2), A(\mathbf{a})) = 0.$$

By nondegeneracy of the standard inner product on $\mathbb{F}^d$, this implies $\mathbf{w}_1 = \mathbf{w}_2$. Hence W is injective. Since $\mathbb{F}^d$ and $\mathbb{E}$ have the same finite cardinality, W is bijective. The same argument shows that A is bijective.

We now prove that W is $\mathbb{F}$ -linear. For $\mathbf{w}_1, \mathbf{w}_2, \mathbf{a} \in \mathbb{F}^d$, linearity of H gives

$$\begin{aligned} & \mu_H(W(\mathbf{w}_1 + \mathbf{w}_2) - W(\mathbf{w}_1) - W(\mathbf{w}_2), A(\mathbf{a})) \\ &= \langle \mathbf{w}_1 + \mathbf{w}_2, \mathbf{a} \rangle_{\mathbb{F}} - \langle \mathbf{w}_1, \mathbf{a} \rangle_{\mathbb{F}} - \langle \mathbf{w}_2, \mathbf{a} \rangle_{\mathbb{F}} \\ &= 0. \end{aligned}$$

Since A is surjective, the second argument ranges over all of $\mathbb{E}$. Nondegeneracy of μ_H therefore implies $W(\mathbf{w}_1 + \mathbf{w}_2) = W(\mathbf{w}_1) + W(\mathbf{w}_2)$. Similarly, for $\lambda \in \mathbb{F}$,

$$\begin{aligned} & \mu_H(W(\lambda \mathbf{w}) - \lambda W(\mathbf{w}), A(\mathbf{a})) \\ &= \langle \lambda \mathbf{w}, \mathbf{a} \rangle_{\mathbb{F}} - \lambda \langle \mathbf{w}, \mathbf{a} \rangle_{\mathbb{F}} \\ &= 0. \end{aligned}$$

Again using surjectivity of A and nondegeneracy of μ_H , we obtain $W(\lambda \mathbf{w}) = \lambda W(\mathbf{w})$. Hence W is $\mathbb{F}$ -linear. By symmetry, A is also $\mathbb{F}$ -linear. Since they are also bijective, W and A are linear isomorphisms. From this it follows that $(W(\mathbf{e}_1), \dots, W(\mathbf{e}_d))$ and $(A(\mathbf{e}_1), \dots, A(\mathbf{e}_d))$ are bases of $\mathbb{E}$. Finally, from $\mu_H(W(\mathbf{e}_i), A(\mathbf{e}_j)) = \delta_{ij}$ it follows they are dual with respect to μ_H . $\square$

Proof of the converse direction. Can be directly shown using a basis expansion. Write $\mathbf{w} = \sum_{i=1}^d w_i \mathbf{e}_i$ and $\mathbf{a} = \sum_{j=1}^d a_j \mathbf{e}_j$. Since W and A are $\mathbb{F}$ -linear, $W(\mathbf{w}) = \sum_{i=1}^d w_i W(\mathbf{e}_i)$ and $A(\mathbf{a}) = \sum_{j=1}^d a_j A(\mathbf{e}_j)$. Therefore,

$$\begin{aligned}
H(W(\mathbf{w})A(\mathbf{a})) &= \mu_H(W(\mathbf{w}), A(\mathbf{a})) \\
&= \sum_{i,j} w_i a_j \mu_H(W(\mathbf{e}_i), A(\mathbf{e}_j)) \\
&= \sum_{i,j} w_i a_j \delta_{ij} \\
&= \sum_i w_i a_i \\
&= \langle \mathbf{w}, \mathbf{a} \rangle_{\mathbb{F}}.
\end{aligned}$$

□

The $n = 1$ case extends naturally to $n > 1$ by doing blockwise packing, i.e. we embed using $W' = I_n \otimes W$ and $A' = I_n \otimes A$ and keep the same H . Let $\mathbf{w} = (\mathbf{w}_1, \dots, \mathbf{w}_n)$ and $\mathbf{a} = (\mathbf{a}_1, \dots, \mathbf{a}_n)$ then

$$\begin{aligned}
H(\langle W'(\mathbf{w}), A'(\mathbf{a}) \rangle_{\mathbb{E}}) &= H\left(\sum_{i=1}^n W(\mathbf{w}_i)A(\mathbf{a}_i)\right) \\
&= \sum_{i=1}^n H(W(\mathbf{w}_i)A(\mathbf{a}_i)) \\
&= \sum_{i=1}^n \langle \mathbf{w}_i, \mathbf{a}_i \rangle_{\mathbb{F}} \\
&= \langle \mathbf{w}, \mathbf{a} \rangle_{\mathbb{F}}.
\end{aligned}$$

Going forward, we use W and A also for their blockwise extensions when the meaning is clear. We do not consider embeddings that mix coordinates across different blocks of size d , as such embeddings are unlikely to be useful in practice.

We have shown that every $n = 1$ embedding with a linear output map is determined by a choice of a nonzero linear map H and linear isomorphism W . The map W then determines the first basis $\beta_i = W(\mathbf{e}_i)$ and the map A is then forced to map to the μ_H -dual basis.

The linearity assumption on the output map is necessary. Indeed, composing a linear embedding with a multiplicative bijection $g : \mathbb{E} \rightarrow \mathbb{E}$ generally produces an embedding that is nonlinear. Whether every nonlinear embedding arises from such a multiplicative reparameterization is a separate question, which we do not need here.

C.3 Properties

Let's consider some general properties of bilinear embeddings.

Theorem C.2 (Preservation of MLE openings). *Given $d = 2^k$, $n = 2^l$ and a $\mathbb{F}$ -linear map $A : \mathbb{F}^d \rightarrow \mathbb{E}$ applied blockwise, then if $\mathbf{a} \in \mathbb{F}^{2^{k+l}}$ given by $a_i = \text{eq}_{\mathbb{F}}(\mathbf{r}, \mathbf{b}(i))$ where $\mathbf{b}$ is the binary encoding, the embedding $A(\mathbf{a}) \in \mathbb{E}^{2^l}$ is given by*

$$(A(\mathbf{a}))_i = \alpha \cdot \text{eq}_{\mathbb{E}}(\mathbf{r}', \mathbf{b}(i))$$

for some $\alpha \in \mathbb{E}$ and $\mathbf{r}' \in \mathbb{E}^l$.

In particular this means that any bilinear map can also be used to lift an multilinear commitment scheme.

Theorem C.3 (Preservation of succinct verification). *If $\mathbf{a}$ has a Matrix-branching program over $\mathbb{F}$ for succinct evaluation of its multilinear extension, then $A(\mathbf{a})$ also has one over $\mathbb{E}$, which is at most d times wider.*

Now assume we have $\mathbf{w} \in \mathbb{F}^{nd}$ but this time we want to open it in extension field weights, e.g. $\mathbf{w} \in \mathbb{E}^{nd}$. Such a situation naturally occurs when doing a base-field sumcheck with extension field challenges.

Pick a basis, expand in d inner products, prover gives values, RLC together, single opening.

C.4 Constructions

The set of nonzero linear output maps H is $\mathbb{E}^* \setminus \{0\}$ which is isomorphic to $\mathbb{E}^\times$. The set of linear isomorphism W is isomorphic to $\text{GL}_d(\mathbb{F})$, hence the total number of bilinear embeddings is $|\mathbb{E}^\times| \cdot |\text{GL}_d(\mathbb{F})|$. We'll now look at specific choices and their properties.

C.4.1 Trace embedding

Let $q = |\mathbb{F}|$, the q -power Frobenius map is $\varphi : \mathbb{E} \rightarrow \mathbb{E}$ given by $x \mapsto x^q$. It is an $\mathbb{F}$ -linear field automorphism of order d , and its fixed field is precisely $\mathbb{F}$:

$$\mathbb{E}^\varphi = \{x \in \mathbb{E} : \varphi(x) = x\} = \mathbb{F}.$$

The field trace is the $\mathbb{F}$ -linear map $\text{Tr}_{\mathbb{E}/\mathbb{F}} : \mathbb{E} \rightarrow \mathbb{F}$ defined by

$$\text{Tr}_{\mathbb{E}/\mathbb{F}}(x) = \sum_{i=0}^{d-1} \varphi^i(x) = \sum_{i=0}^{d-1} x^{q^i}.$$

To see that this sum is indeed in $\mathbb{F} \subseteq \mathbb{E}$ observe the sum is fixed by φ , since

$$\varphi \left(\sum_{i=0}^{d-1} \varphi^i(x) \right) = \sum_{i=1}^d \varphi^i(x) = \sum_{i=0}^{d-1} \varphi^i(x),$$

where $\varphi^d(x) = \varphi^0(x)$. Hence the trace lies in the fixed field $\mathbb{F}$.

This makes $\text{Tr}_{\mathbb{E}/\mathbb{F}}$ a natural choice of output map H . More generally, every $\mathbb{F}$ -linear map $H : \mathbb{E} \rightarrow \mathbb{F}$ is uniquely of the form $H(x) = \text{Tr}_{\mathbb{E}/\mathbb{F}}(\alpha x)$ for some $\alpha \in \mathbb{E}$. Consequently, the nonzero output maps are parameterized by $\mathbb{E}^\times$.

The Galois orbit embedding chooses the basis $\beta_i = \varphi^i(\gamma)$ which has dual $\beta_i^\vee = \varphi^i(\gamma^\vee)$.

Ulrich: This sounds to me very much related to Galois Fribinius on an inner product with a general covector. You would end up with the multi-claim of the inner products of the packed poly with the Galois orbit of the covector. This is how I view it: Given $\mathbf{p}_1, \dots, \mathbf{p}_d \in \mathbb{F}^n$, and $\mathbf{q}_1, \dots, \mathbf{q}_d \in \mathbb{F}^n$, we are interested in

$$\sum_{k=1}^d \langle \mathbf{p}_k, \mathbf{q}_k \rangle,$$

right?

The packing map sends the coeff vectors $\mathbf{p}_1, \dots, \mathbf{p}_d$ to $\mathbf{p} = \beta_1 \cdot \mathbf{p}_1 + \dots + \beta_d \cdot \mathbf{p}_d$, where $\beta_1, \dots, \beta_d$ is an $\mathbb{F}$ -basis of $\mathbb{E}$. Choose the element a_k in $\mathbb{E}$ which extracts the k -th coordinate

$$\mathbf{p}_k = \Phi_k(\mathbf{p}) = \text{Tr}_{\mathbb{E}/\mathbb{F}}(a_k \cdot \mathbf{p}).$$

Then

$$\begin{aligned} \langle \mathbf{p}_k, \mathbf{q}_k \rangle &= \left\langle \sum_{i=0}^d \varphi^i(a_k \cdot \mathbf{p}), \mathbf{q}_k \right\rangle \\ &= \sum_{i=0}^d \varphi^i(a_k) \cdot \varphi^i(\langle \mathbf{p}, \mathbf{q}_k \rangle), \end{aligned}$$

since $\mathbf{q}_k$ is over $\mathbb{F}$. This reduces our inner product claim to that of the packed poly $\mathbf{p}$ with the unpacked polys $\mathbf{q}_1, \dots, \mathbf{q}_k$.

But does that match our use case? We are rather interested in $\langle \mathbf{p}_k, \mathbf{v} \rangle$ for some $\mathbf{v} \in \mathbb{E}^n$. In this case,

$$\begin{aligned} \langle \mathbf{p}_k, \mathbf{v} \rangle &= \left\langle \sum_{i=0}^d \varphi^i(a_k \cdot \mathbf{p}), \mathbf{q}_k \right\rangle \\ &= \sum_{i=0}^d \varphi^i(a_k) \cdot \varphi^i(\langle \mathbf{p}, \varphi^{-i}(\mathbf{v}) \rangle), \end{aligned}$$

and we end up with inner products of the packed poly with the Galois orbit of $\mathbf{v}$.

C.4.2 Coefficient projection

In the coefficient projection we assume a concrete representation of $\mathbb{E}$ as $\mathbb{F}[X]/(f(X))$ for some monic degree d polynomial f irreducible over $\mathbb{F}$. We assume f is chosen for implementation efficiency: it is sparse with mostly low order terms. Further we represent members of $\mathbb{E}$ in the monomial basis $(1, X, X^2, \dots, X^{d-1})$. We can then represent W and A by invertible matrices in $\mathbb{F}^{d \times d}$ and H by a vector in $\mathbb{F}^d$. For implementation efficiency we seek an embedding with sparse W and A . It turns out taking the most efficient choices for H and W also gives efficient A :

Theorem C.4. *Write $f(X) = f_d X^d + f_{d-1} X^{d-1} + \dots + f_1 X + f_0$ so that $f_d = 1$ and $f_0 \neq 0$. Take $W = I_d$ and $H = (1, 0, \dots, 0)$. Then the dual-basis embedding A is given by*

$$A_{ij} = \begin{cases} 1, & i = j = 0, \\ -f_{i+j}/f_0, & i, j \geq 1 \text{ and } i + j \leq d, \\ 0, & \text{otherwise.} \end{cases}$$

In particular, A is a bordered upper anti-triangular Hankel matrix. For $d = 5$ it looks like

$$A = \frac{1}{f_0} \begin{pmatrix} f_0 & 0 & 0 & 0 & 0 \\ 0 & -f_2 & -f_3 & -f_4 & -1 \\ 0 & -f_3 & -f_4 & -1 & 0 \\ 0 & -f_4 & -1 & 0 & 0 \\ 0 & -1 & 0 & 0 & 0 \end{pmatrix}.$$

Notably f_1 does not appear and for $k > 1$ the f_k coefficient appears $k - 1$ times. This perfectly matches the usual implementation preference for sparse reduction polynomials with mostly low-order terms.

For the 127-bit field $\mathbb{F}_2[X]/(X^{127} + X + 1)$ the A map is simply a reversal of the $i > 0$ coordinates. For the GHASH field $\mathbb{F}_2[X]/(X^{128} + X^7 + X^2 + X + 1)$ the map additionally needs seven XOR corrections.